UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Calidad de Software

Mag. Pablo Virgolini

24 de agosto de 2026

¿Recordamos?



¿Qué es el testing en software?

¿Qué es el Testing?

Disciplina fundamental de la ingeniería de software cuyo objetivo es encontrar defectos antes de que lleguen a los usuarios. No se hace solo al final, se ejecuta en paralelo a todo el ciclo de vida.

Objetivos principales:

1. Verificar el cumplimiento de requisitos funcionales y no funcionales
2. Garantizar la usabilidad para los usuarios finales
3. Asegurar la seguridad y protección de datos
4. Verificar el rendimiento y la estabilidad
5. Detectar defectos lo antes posible en el ciclo de desarrollo

Necesidad de realizar pruebas



Después, recién, el modelo formal:

Error	Defecto	Fallo
Lo comete una persona.	Se incorpora al código.	Se manifiesta en la ejecución.

Los problemas pueden aparecer en distintos momentos del ciclo de desarrollo.
No siempre se detectan en la misma etapa.

Error, defecto y falla

ERROR

Acción **humana** incorrecta. Ocurre en la mente de una persona: un analista, diseñador o desarrollador.

El error ya ocurrió en el pasado. No se detecta con testing. Luego se investiga su causa raíz para evitar que vuelva a ocurrir.

DEFECTO

El error queda incorporado al software (edad >= 16 en vez de >= 18). Puede permanecer oculto meses.

Se puede detectar de dos formas:

- Técnicas estáticas
- Testing dinámico

FALLO

El defecto se manifiesta de forma visible durante la ejecución: el sistema aprueba lo que no debía.

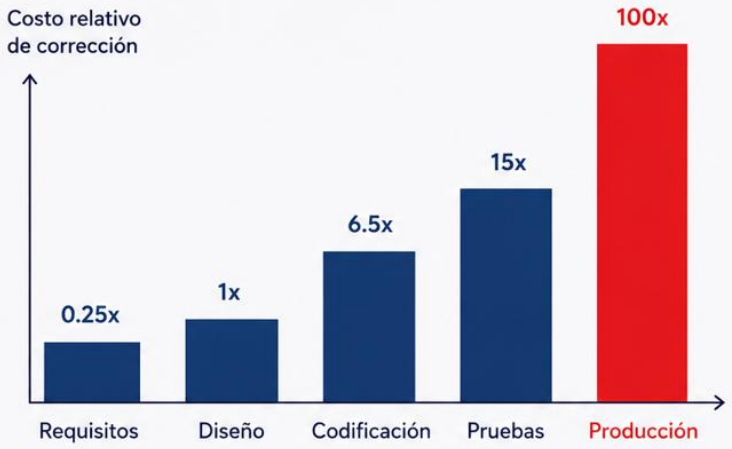
Lo detecta el usuario o los tests durante la ejecución (síntoma observable).



Ejemplo — una calculadora

El desarrollador olvida validar la división por cero. Con 8/2, 15/3, 20/5 todo funciona: el defecto sigue oculto. Semanas después alguien ingresa **15/0** y la aplicación falla. El defecto existía desde el principio; lo nuevo fue el fallo.

Impacto en costos



- **Corregir un defecto después de la entrega puede ser ~100 veces más caro que durante el diseño en proyectos grandes.** En proyectos chicos es **~5 veces.**

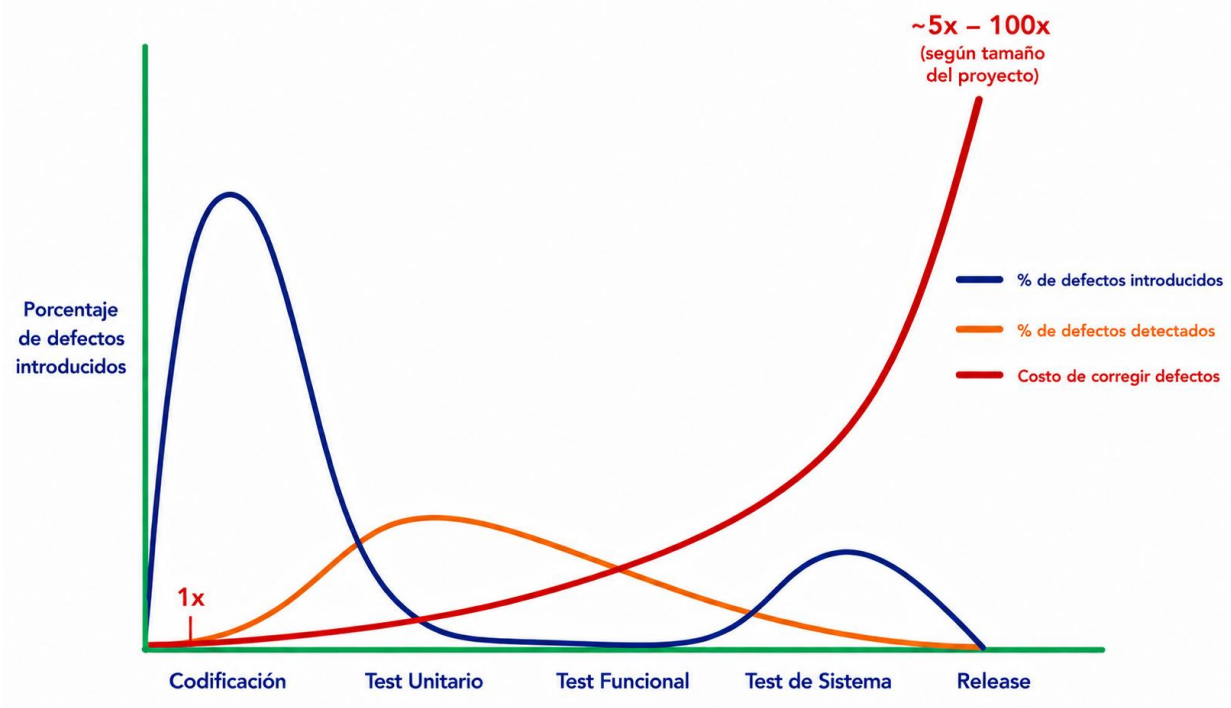
- Un bug corregido en la fase de requisitos cuesta **prácticamente una conversación.** El mismo error corregido después del release puede implicar rediseño, recodificación, retesteo, documentación y un parche en producción.

La prevención y la detección temprana ahorran tiempo, esfuerzo y dinero. Invertir en revisión de requisitos, diseño y pruebas tempranas es la mejor estrategia de calidad.

Cada defecto que llega a producción no solo **cuesta más** corrgirlo, también impacta en la experiencia del cliente y en la reputación del producto.

Fuente: https://budgetoverrun.com/cost-of-change-curve

Impacto en costos



La recomendación es mover la detección de bug lo más temprano posible mediante tests automatizados, revisiones de código y análisis estático. Ahí es donde la curva de costo todavía está plana.

Siete Principios del Proceso de Pruebas

Estos principios fundamentales establecen la base conceptual de todas las metodologías de pruebas.



Caso: Incidente VSE

▶ Intro
Te invito a leer el caso de la Vancouver Stock Exchange (VSE) .

▶ Mecanismo del Bug
El sistema truncaba el índice a 3 decimales después de cada recálculo, en vez de conservar la precisión completa y redondear recién al mostrar el valor final. Como cada recálculo partía del valor ya truncado (y no del real), el error no se descartaba: se arrastraba al siguiente cálculo, siempre en la misma dirección.

▶ Pregunta
¿Por qué un error tan "chico" como un truncamiento en cada paso puede hacer que un índice pierda el 70% de su valor real?

Trabajamos



Apliquemos los principios fundamentales del testing al caso del incidente en la VSE.

Cuando hayan terminado, utilicen una IAG y comparen los resultados

Modelos de desarrollo de software

En todo modelo existen dos principios comunes: cada actividad debe ser probada, y las pruebas empiezan desde el inicio, no al final.

Secuencial (cascada)

Planificación rígida: cada fase se completa antes de comenzar la siguiente. Pruebas mas oncentradas al final. Ideal para requisitos estables.

Iterativo e incremental

Desarrollo en incrementos funcionales entregados parcialmente. Pruebas por iteración: mayor adaptabilidad y detección temprana.

Ágil (Scrum)

Sprints cortos con entrega continua de valor. Pruebas continuas y regresión frecuente. Centrado en colaboración y respuesta al cambio.

¿Qué modelo recomendarían?

A

Sistema de gestión universitaria

Permite gestionar estudiantes, inscripciones, materias, cursado, exámenes, calificaciones y emisión de certificados.

Modelo recomendado
Desarrollo Iterativo e Incremental

Permite incorporar funcionalidades en incrementos sucesivos y, en cada iteración, revisar y mejorar lo ya desarrollado a partir de pruebas y feedback.

B

Plataforma de aprendizaje en línea

Ofrece cursos virtuales, clases en vivo, evaluaciones automáticas, seguimiento del progreso y emisión de certificados.

Modelo recomendado
Desarrollo Ágil (Scrum)

Necesita adaptarse rápido al feedback de usuarios y cambiar requisitos con frecuencia.

C

Sistema de gestión de logística

Coordina rutas de entrega, seguimiento de envíos en tiempo real, gestión de almacenes y notificaciones.

Modelo recomendado
Desarrollo Ágil (Scrum)

Entorno dinámico y cambiante que requiere entregas frecuentes y mejora continua.

D

Sistema de gestión telemedicina

Permite consultas médicas virtuales, agendamiento de turnos, historia clínica electrónica y recetas digitales.

Modelo recomendado
Cascada

Requiere una planificación secuencial, con definiciones claras de requisitos y entregas en etapas.

Recomendación general

Elegir el modelo de desarrollo depende de la estabilidad de los requisitos, el nivel de incertidumbre y las regulaciones del proyecto. **Ágil es ideal para entornos cambiantes e innovadores; Incremental para sistemas complejos con entregas por etapas; Cascada para proyectos con requisitos fijos y alta exigencia regulatoria.**

Niveles de prueba

Las pruebas de software se estructuran en niveles que abarcan desde componentes individuales hasta la validación completa del sistema.



Tipos de pruebas

Pruebas Funcionales

Verifican que el software cumple con los requisitos funcionales especificados.

Exploratorias
Diseño y ejecución simultáneos.

Regresión
Verifican que cambios no afecten funcionalidades

Compatibilidad
Funcionamiento en distintos entornos.

Smoke
Verificación rápida de funcionalidades críticas.

Sanity
Comprobación específica tras una corrección.

Pruebas No Funcionales

Evalúan atributos de calidad más allá de la funcionalidad.

Carga / rendimiento
Comportamiento bajo alta demanda.

Recuperación
Capacidad para recuperarse tras fallos.

Instalación
Proceso de instalación correcto.

Estructurales
Análisis del código interno.

Configuración
Comportamiento con distintas configuraciones.

ISO/IEC 25010: Modelo de Calidad

Estándar internacional que define ocho características clave para evaluar la calidad del software.

1

Adecuación funcional

Las funciones cumplen los requisitos. Ej: WhatsApp envía mensajes correctamente.

2

Eficiencia de desempeño

Rendimiento vs. recursos usados. Ej: Netflix carga rápido con conexión lenta.

3

Compatibilidad

Intercambio con otros sistemas. Ej: Office abre distintos formatos.

4

Usabilidad

Facilidad de aprendizaje y uso. Ej: Instagram es intuitivo para cualquier edad.

5

Fiabilidad

Desempeño constante. Ej: Google Maps funciona sin fallos.

6

Seguridad

Protección de información. Ej: apps bancarias con múltiples capas de autenticación.

7

Mantenibilidad

Facilidad para modificar. Ej: Visual Studio Code permite agregar extensiones nuevas sin modificar el núcleo.

8

Portabilidad

Se transfiere entre entornos. Ej: Spotify funciona igual en móvil, web y escritorio.

Calidad Interna/Externa y en Uso

Calidad Interna/Externa

Aspectos técnicos del software que pueden medirse objetivamente:

- Métricas de código (complejidad, duplicación)
- Cobertura de pruebas automatizadas
- Número de defectos por línea de código
- Tiempo de respuesta controlado
- Cumplimiento de estándares de programación

Calidad en Uso

Centrada en la experiencia del usuario final con el producto:

- Facilidad para completar tareas específicas
- Satisfacción del usuario
- Prevención de errores del usuario
- Accesibilidad para diferentes perfiles
- Eficiencia percibida al utilizar el sistema

Analogía: un auto puede tener un motor técnicamente impecable (calidad interna) pero si los asientos son incómodos y los controles confusos (mala calidad en uso), la experiencia general será negativa.

CASO DE ESTUDIO



Knight Capital (2012)

Caso: Knight Capital (2012)

▶ Intro

Te invito a leer el caso de la Knight Capital (2012).

▶ Mecanismo del Bug

Para incorporar una nueva funcionalidad en su sistema de enrutamiento de órdenes, Knight Capital debía actualizar el software en ocho servidores. Sin embargo, el despliegue se realizó correctamente solo en siete. El servidor que quedó sin actualizar conservaba una versión anterior del sistema y, al recibir una bandera reutilizada para la nueva funcionalidad, activó por error una rutina antigua que todavía permanecía en producción. Como consecuencia, comenzó a generar órdenes bursátiles incorrectas de manera automática y a gran velocidad.

Caso: Knight Capital (2012)

En agosto de 2012, Knight Capital debía incorporar una nueva funcionalidad en su sistema de enrutamiento de órdenes. Para activarla se reutilizó una bandera que anteriormente ejecutaba una rutina denominada Power Peg, que ya no se utilizaba pero cuyo código permanecía en producción.

Durante el despliegue, uno de los ocho servidores no fue actualizado. Mientras siete ejecutaron correctamente la nueva versión, el servidor restante interpretó la bandera según la lógica anterior y reactivó el código obsoleto, que comenzó a generar órdenes de manera incorrecta.



45 minutos

Duración del incidente antes de poder detenerlo



4 millones

Órdenes enviadas al mercado (se esperaban solo 212)



USD 440-457M

Pérdida de trading estimada



USD 400M + 12M

Capital de emergencia recibido y multa posterior de la SEC



Señal ignorada: un sistema interno generó 97 correos automáticos de error antes de la apertura del mercado, pero no estaban diseñados como alertas operativas y no dispararon una respuesta a tiempo.

Trabajamos



Knight Capital y las 8 características de ISO/IEC 25010

Para cada característica: indiquen si se vio comprometida (Sí / No / No Sabe), con qué evidencia concreta del caso y qué práctica la hubiera prevenido.

Elijan las 3 características más determinantes en la falla, ordenadas de mayor a menor impacto, y justifiquen el orden.

Knight Capital y las 8 características de ISO/IEC 25010

Característica	¿Comprometida?	Evidencia concreta del caso	Práctica que la hubiera prevenido
Adecuación funcional			
Eficiencia de desempeño			
Compatibilidad			
Usabilidad			
Fiabilidad			
Seguridad			
Mantenibilidad			
Portabilidad			

Top 3 más determinantes (orden de mayor a menor impacto): 1) _____ 2) _____ 3) _____